RATG-OC: Risk-Adaptive Temporal Gating and Track-Level OCR Consensus for Efficient Traffic Surveillance

Chandwale Zarkaan, Tejas R Mutalik, Umar Munshi, Dr. Karthik P
Department of Information Science and Engineering
Cambridge Institute of Technology
Bengaluru, India

***Abstract*—Conventional automated traffic monitoring pipelines execute heavy computer vision models—specifically deep object detectors and text recognizers—on every video frame at a fixed rate. In typical surveillance streams, this burns compute on static backgrounds and passes motion-blurred vehicle crops to the recognizer, yielding noisy and unstable plate records. We introduce RATG-OC (Risk-Adaptive Temporal Gating and Track-Level OCR Consensus), a drop-in scheduling layer that wraps existing YOLOv8, DeepSORT, and EasyOCR pipelines without modifying model weights. RATG-OC evaluates an interpretable scalar risk score for each incoming frame based on background pixel displacement, Kalman tracker covariance growth, boundary proximity to enforcement lines, and track-level plate confidence. Low-risk frames bypass deep detection entirely, updating active tracks through linear motion projection. Vehicle crops undergo OCR only after satisfying a Laplacian sharpness criterion, with recognized strings pooled across frames into a track-indexed voting buffer to resolve a single consensus identifier. Evaluated on a 720p, 30-FPS intersection recording, RATG-OC lifts throughput from 8.7 to 18.9 frames per second, cuts mean per-frame latency from 114.9 ms to 52.9 ms, and reduces OCR inferences by 71.8%. Temporal consensus filtering also improves exact license plate recognition from 84.3% (70/83) to 89.2% (74/83), while vehicle detection mAP@0.5 remains steady at 90.4%.**

***Index Terms*—intelligent transportation systems, adaptive inference, vehicle tracking, automatic number-plate recognition, edge video analytics**

I. Introduction

Fixed-mount traffic cameras record hours of visually redundant footage. Most frames show static asphalt or vehicles idling during red lights, yet within this monotony lie time-sensitive enforcement events—such as a motorcycle bypassing a stop bar or a vehicle plate visible only for brief spans between occlusions. Standard municipal deployments run fixed-rate inference loops comprising a detector (e.g., YOLOv8), an association tracker (e.g., DeepSORT), an optical character recognition engine (e.g., EasyOCR), and rule-checking logic. Because every frame undergoes the same exhaustive processing, compute demand scales strictly with video duration rather than dynamic activity in the roadway.

Recent detection architectures achieve impressive latency-accuracy trade-offs [1]–[4], and modern tracking methods improve trajectory continuity through refined motion priors and low-confidence matching [5]–[8]. Dedicated license plate recognition models likewise demonstrate resilience against optical skew and dirty plates [9]–[12]. However, optimizing these stages independently leaves system-level scheduling unaddressed. In production, two operational questions dominate: *when can the pipeline safely skip a detector pass, and when does a tracked vehicle warrant an OCR call?*

RATG-OC addresses both questions through a lightweight risk governor. When the scene is calm, track uncertainties remain tight, and recognized plates are already established, the system suppresses the detector and advances bounding boxes using Kalman predictions. When a vehicle nears an enforcement zone, tracker covariance spikes, or plate readings remain unverified, full detection resumes within a single frame. Because RATG-OC functions as a supervisory controller, it requires no model retraining and integrates directly into existing Python-based deployment architectures.

We outline three primary contributions:

- A multi-factor risk governor that balances background pixel differences, Kalman filter covariance, violation boundary proximity, detector age, and plate evidence to dynamically gate detector and OCR invocations.
- A track-indexed consensus cache that weights text candidates by recognition probability, image sharpness, and syntax validity, eliminating transient OCR misreads.
- An experimental evaluation on a 720p intersection recording, confirming a 2.17× speedup and a 71.8% drop in OCR load with negligible impact on detection accuracy (0.6% mAP drop) and a 4.9 percentage-point gain in exact plate reading accuracy (resolving 74 versus 70 of 83 readable plates).

II. Related Work

Automated traffic enforcement pipelines link multiple computer vision models into an end-to-end processing chain. Here we review prior work spanning real-time object detection, multi-object tracking, automated license plate recognition (ALPR), integrated violation enforcement, and dynamic video analytics.

A. Real-Time Vehicle Detection

Modern traffic analytics largely rely on one-stage convolutional or transformer-based detectors. Wang *et al.*'s YOLOv7 [1] introduced re-parameterized convolutions and auxiliary

head supervision to maximize edge throughput, while Bie *et al.* [2] tailored YOLOv5n with coordinate attention and separable convolutions specifically for lightweight roadside hardware. More recently, transformer models such as RT-DETR [3] and NMS-free designs like YOLOv10 [4] eliminate post-processing bottlenecks by directly outputting non-overlapping bounding boxes. While these advances steadily compress the per-frame inference budget, running them blindly on every incoming video frame remains wasteful during low-traffic periods, nor do they determine when downstream recognition models should be triggered.

B. Multi-Object Tracking

Tracking vehicle detections across time is necessary to calculate speed and observe red-light incursions. ByteTrack [5] shows that recovering low-confidence detection boxes reduces false track terminations during brief occlusions. In parallel, OC-SORT [6] modifies the standard Kalman filter to account for accumulated momentum when tracks temporarily disappear. Modernizing classic tracking paradigms, StrongSORT [8] refines DeepSORT using stronger appearance feature extractors and dual-direction association, while MOTRv2 [7] uses external anchor detections to guide transformer-based tracking queries. Yet virtually all tracking benchmarks operate under the premise that detector outputs arrive every frame. In edge deployments, invoking high-dimensional ReID embeddings and heavy association algorithms continuously quickly saturates the onboard GPU.

C. License-Plate Recognition

License plate recognition (ALPR) in real-world intersections must handle steep camera angles, rain glare, and non-standard typography. Shafi *et al.* [9] combine morphological filtering with neural character classifiers to read non-standard plates under inconsistent ambient illumination. Ke *et al.* [10] integrate spatial transformer networks with monocular depth estimation to rectify skewed plates while estimating vehicle approach velocities. For unconstrained roadway environments, Wei *et al.* [11] develop an anchor-free network that localizes plates and reads text in a unified pass, while Zhang *et al.* [12] connect transformer-based OCR sequences directly with vehicle fleet emission models. These studies concentrate primarily on single-crop character recognition accuracy rather than temporal integration; a blurred reading from a distant vehicle is typically given the same operational weight as a sharp, centered capture.

D. Traffic Analytics and Violation Reasoning

Translating trajectories into automated citations requires checking bounding boxes against spatial boundary rules. Ong *et al.* [13] build an integrated vision pipeline to detect illegal lane changes, red-light runs, and speeding using standard vision backbones. Fahrezi and Laksana [14] pair YOLOv8 and ByteTrack specifically to monitor urban queue densities and signal adherence. Similar end-to-end architectures have been developed for helmet enforcement [15], combining head-region localization, helmet classification, and license-plate extraction into automated ticketing pipelines. Across wide-area networks, Yang *et al.* [16] distribute vehicle re-identification and tracking between edge nodes and centralized cloud servers. These applied systems, however, treat rule verification and inference scheduling in isolation: a camera running at full frame rate expends the same processing power when vehicles are stationary as when an infraction is actively occurring.

E. Adaptive Edge Video Analytics

To reduce unnecessary computation on static footage, adaptive analytics dynamically throttle pipeline workloads. FrameHopper [17] trains a reinforcement learning agent to predict how many frames can be safely skipped without missing target objects. Approaching redundancy at the spatial level, PASS [18] identifies and bypasses background pixel patches that remain unchanged between frames. Dynamic resolution schemes [19] and partition frameworks such as SplitStream [20] further distribute feature computation across edge and server hardware according to network conditions. While these approaches confirm that selective skipping cuts processing overhead, their control policies optimize generic object detection rather than legal compliance. None of them couple frame skipping with tracker covariance, stop-line boundary proximity, or multi-frame plate voting. RATG-OC bridges this exact gap by formulating inference as an adaptive risk-gated process.

III. Proposed Methodology and Architecture

A. System Model

Let $I_t \in \mathbb{R}^{H\times W\times 3}$ denote the video frame captured at discrete time index t. An unmodified baseline pushes every frame sequentially through the detector D, the tracker T, the plate reader O, and the violation module V. RATG-OC decomposes execution into two operational regimes:

1) **Primary Mode:** The pipeline runs detector D followed by tracker T with updated bounding boxes and visual features.
2) **Propagate Mode:** The pipeline skips D, supplies an empty detection set to T, and allows the Kalman filter to advance existing tracks along their current trajectory vectors while preserving their cached identities and classifications.

Rule-checking module V runs unconditionally on every frame, so speed computations and stop-line boundary checks never miss an event.

The choice of mode on frame t is governed by a scalar risk index $r_t \in [0, 1]$:

$$r_t = \mathrm{clip}_{[0,1]}(w_d d_t + w_u u_t + w_z z_t + w_a a_t + w_q q_t). \quad (1)$$

The terms in Eq. (1) quantify distinct drivers of perceptual urgency:

- d_t measures mean absolute pixel differences between downscaled 160×120 grayscale thumbnails of I_t and

I_{t-1}; an empty road yields $d_t \approx 0$, whereas new vehicles entering the frame elevate this value.

- u_t computes the mean normalized trace of active Kalman covariance matrices, rising as tracks coast without fresh measurement updates.
- z_t tracks the inverse spatial distance from any vehicle bounding box to the nearest configured enforcement boundary (e.g., stop line or speed trap corridor).
- $a_t = \min\big(1,\ (t - t_D)/K_{\max}\big)$ tracks elapsed frames since the last detector pass at t_D, reaching 1.0 at hard ceiling $K_{\max}$.
- q_t represents the proportion of active tracks lacking a locked plate reading.

All weights are non-negative and normalized such that $\sum w = 1$. In our benchmark, hyperparameters are set to $w = [0.25, 0.20, 0.25, 0.15, 0.15]$, $\tau_D = 0.5$, $\tau_Z = 0.7$, $\tau_L = 100$, $\tau_C = 2.0$, $m = 0.5$, and cache limit $B = 10$.

As formulated in Eq. (2), a primary detection step fires whenever risk surpasses threshold τ_D or safety overrides trigger:

$$g_t = \mathbf{1}[r_t \geq \tau_D \ \vee \ (t - t_D) \geq K_{\max} \ \vee \ z_t \geq \tau_Z]. \tag{2}$$

A small hysteresis margin $\delta > 0$ avoids boundary jitter: once triggered, risk must fall below $\tau_D - \delta$ before returning to propagate mode.

B. Selective OCR and Temporal Consensus

Rather than processing every vehicle crop on every frame, OCR runs selectively. A crop from track i is dispatched to recognizer O only if track i's plate is still unresolved and the crop's Laplacian variance exceeds blur threshold τ_L.

Each candidate text string $s_{i,j}$ extracted on track i receives a confidence weight $\omega_{i,j}$ calculated by Eq. (3):

$$\omega_{i,j} = c_{i,j}^{\alpha} \log(1 + h_{i,j})[\lambda + (1 - \lambda)v_{i,j}]. \tag{3}$$

Here $c_{i,j}$ is OCR recognition confidence, $h_{i,j}$ is Laplacian edge sharpness, and $v_{i,j} \in \{0, 1\}$ checks adherence to standard regional registration patterns (e.g., two letters, two digits, two letters, four digits). Strings are normalized to uppercase alphanumerics, and candidates within Levenshtein distance one cluster into equivalence group C. The lifetime consensus plate $\hat{s}_i$ is selected via Eq. (4):

$$\hat{s}_i = \arg\max_{C} \sum_{j : s_{i,j} \in C} \omega_{i,j}. \tag{4}$$

A plate is permanently locked for track i once winning cluster C exceeds weight threshold τ_C and leads runner-up clusters by stability margin m. Once locked, further OCR calls on that vehicle cease entirely.

C. Algorithm and Complexity

Algorithm 1 specifies the complete frame handling routine.

The computational overhead added by the RATG-OC controller is bounded by $O(P_s + N_t B)$, where $P_s = 160 \times 120$ pixels for the thumbnail difference, N_t is active tracks, and B is the maximum cache size per track (capped at $B = 10$). In practice, evaluating this logic requires under 1.2 ms on the CPU, which is negligible compared to the 32 ms detector pass or 25 ms OCR inference.

Algorithm 1 RATG-OC Frame Processing Pipeline

Require: Frame I_t, tracks $\mathcal{T}$, caches $\mathcal{C}$, parameters $K_{\max}$, τ_D, τ_Z

Compute d_t, u_t, z_t, a_t, q_t and composite risk r_t via Eq. (1)
if $r_t \geq \tau_D$ **or** $(t - t_D) \geq K_{\max}$ **or** $z_t \geq \tau_Z$ **then**
  $\mathcal{B} \leftarrow D(I_t)$; $\mathcal{T} \leftarrow T(\mathcal{B}, I_t)$; $t_D \leftarrow t$
else
  $\mathcal{T} \leftarrow T(\varnothing, I_t)$ using Kalman motion update
end if
for all confirmed track $i \in \mathcal{T}$ **do**
  if cache $\mathcal{C}_i$ unresolved **and** crop sharpness $h_i \geq \tau_L$ **then**
    $(s, c) \leftarrow O(\text{crop}(I_t, i))$
    Append weighted vote (s, c, h, v) to $\mathcal{C}_i$ via Eq. (3)
    Update consensus plate $\hat{s}_i$ via Eq. (4)
  end if
  Evaluate violation logic $V(i, \hat{s}_i)$ and record new events
end for
return updated tracks $\mathcal{T}$, consensus plates $\hat{s}$, and violations

IV. Experimental Setup and Performance Evaluation

A. Testbed and Evaluation Protocol

To evaluate RATG-OC under realistic municipal deployment constraints, both the baseline and gated systems were benchmarked on a single workstation equipped with an Intel Core i7-12700 CPU, 16 GB DDR5 RAM, and an NVIDIA GeForce RTX 3060 GPU (12 GB VRAM), running Ubuntu 22.04 and Python 3.10. Both pipelines employ identical core models: YOLOv8n pretrained on COCO for vehicle detection, DeepSORT with MobileNet-v2 visual embeddings (`max_age` = 30, `n_init` = 3) for multi-target tracking, and EasyOCR (English configuration) for text recognition at 1280×720 resolution.

The test sequence consists of a 900-frame (30-FPS, 30-second) recording of a four-lane signalized urban intersection during daytime traffic. The scene features a dense mix of passenger cars, motorcycles, city buses, and three-wheeled auto-rickshaws navigating signal phase transitions. Ground truth was established by manual frame-by-frame verification: 127 unique vehicle trajectories, 83 clearly readable license plates, and 14 confirmed infractions (9 red-light incursions, 3 speed violations, and 2 unhelmeted motorcyclists). We measure frame throughput (FPS), mean and 95th-percentile frame latency, peak resident memory, detector trigger percentage, OCR calls per 100 track-frames, vehicle detection mAP@0.5, multi-object tracking IDF1, exact license plate string accuracy, and macro-averaged infraction F1 score.

TABLE I
SYSTEM PERFORMANCE ON 720P URBAN INTERSECTION VIDEO

Metric	Baseline Pipeline	RATG-OC (Proposed)
Throughput (frame/s)	8.7	18.9
Mean latency (ms/frame)	114.9	52.9
P95 latency (ms/frame)	181.2	91.6
Peak memory (GB)	2.84	2.31
Detector invocations (% frames)	100.0	43.6
OCR calls (% track-frames)	100.0	28.2
Vehicle mAP@0.5 (%)	91.0	90.4
Tracking IDF1 (%)	82.6	82.1
Exact plate recognition (%)	84.3	89.2
Violation detection F1	0.781	0.793

B. Results

Table I provides a head-to-head comparison between the baseline and RATG-OC pipelines on the 900-frame benchmark.

The default ungated pipeline processes 8.7 frames per second, creating an acute latency bottleneck where per-frame latency averages 114.9 ms and reaches 181.2 ms at the 95th percentile. Profiling shows this time is consumed almost entirely by full-frame YOLOv8 forward passes ($\sim$32 ms) and repeated EasyOCR inferences on vehicle crops ($\sim$25 ms per crop, with three crops evaluated per frame on average). Under RATG-OC, throughput climbs to 18.9 FPS—a 2.17$\times$ acceleration—while mean latency drops to 52.9 ms and P95 latency falls to 91.6 ms.

The performance gains derive from two operational mechanisms:

1) **Detector Gating:** The composite risk index determines that 56.4% of incoming frames exhibit low motion variance and minimal tracker uncertainty. Bypassing the detector on these frames reduces YOLOv8 execution to 43.6% of total video frames.
2) **Consensus Pruning:** Once a track achieves a stable consensus plate, subsequent OCR calls on that vehicle are suppressed. This policy cuts total OCR calls by 71.8%, dropping from 100 calls per 100 track-frames to 28.2.

These efficiency gains preserve analytical accuracy. Vehicle mAP@0.5 shifts by only 0.6 percentage points (91.0% to 90.4%) and tracking IDF1 drops by 0.5 points (82.6% to 82.1%), as short Kalman propagation intervals (2–4 frames) do not drift significantly before the next primary detection. Exact license plate recognition accuracy improves from 84.3% to 89.2% (correctly identifying 74 of 83 readable vehicles compared to 70 for the baseline). The temporal consensus filter successfully rejects outlier misreadings caused by momentary headlight glare or motion blur that would otherwise corrupt the final record. Consistent multi-frame plate consensus also boosts overall violation detection F1 from 0.781 to 0.793 by preventing duplicate or fragmented infraction records.

C. Ablation Experiments and Operational Edge Cases

To quantify the standalone impact of each mechanism, we evaluated two ablation variants:

- **Gating Alone (No OCR Consensus):** Bypasses both detector and crop-extraction steps on propagate frames, reaching 18.2 FPS; however, plate accuracy stays at 84.3% (70/83) because active frames still rely on unmitigated single-crop readings.
- **Consensus Alone (No Gating):** Suppresses redundant OCR calls but keeps the 32 ms detector active on every frame, reaching 11.2 FPS with 89.2% plate accuracy (74/83).

Both components are thus essential: adaptive gating delivers real-time throughput, while temporal consensus maintains high-precision character readings.

We also examined maximum detector age $K_{\max} \in \{2, 4, 6, 8\}$. Setting $K_{\max} = 2$ achieves 14.1 FPS with zero observable tracking drift. Expanding $K_{\max} = 6$ increases throughput to 21.4 FPS, but introduces occasional identity switches in dense queues where two adjacent motorcycles coast along parallel trajectories. The combination of $K_{\max} = 4$ and risk threshold $\tau_D = 0.5$ provides the optimal trade-off for 30-FPS streams and forms the primary configuration benchmarked in Table I.

Qualitative inspection reveals two edge-case behaviors:

- **Specular Glare at Night:** Direct vehicular headlights can overexpose plate crops, dropping Laplacian sharpness below threshold τ_L. While the filter appropriately rejects these corrupted crops, a vehicle exposed to continuous glare throughout its trajectory will fail to register a consensus plate. Incorporating an active license-plate localization model prior to cropping would improve resilience here.
- **Close-Proximity Identity Swaps:** In tightly clustered stop-line queues, DeepSORT occasionally exchanges track IDs between adjacent two-wheelers. When an ID swap occurs, candidates from two distinct license plates enter the same voting cache. RATG-OC mitigates this by invalidating the consensus buffer whenever a track's spatial velocity vector changes discontinuously.

V. CONCLUSION AND FUTURE WORK

Continuous fixed-rate inference on high-resolution surveillance video represents an inefficient use of computational resources. RATG-OC introduces an adaptive risk-scoring governor and a temporal consensus cache that wraps existing YOLOv8, DeepSORT, and EasyOCR pipelines. Across a 720p urban intersection benchmark, the system reduces detector calls by 56.4% and OCR calls by 71.8%, increasing throughput from 8.7 to 18.9 FPS and reducing average latency by more than half. Concurrently, multi-frame voting elevates exact plate recognition accuracy from 84.3% to 89.2% (74 of 83 plates) with negligible tracking loss.

Planned extensions include deploying the framework to resource-constrained NVIDIA Jetson edge devices, replacing

the heuristic helmet-detection check with a specialized head-region convolutional classifier, and fine-tuning vehicle detection models on regional datasets to better distinguish regional vehicular categories such as motorized auto-rickshaws.

REFERENCES

[1] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 7464–7475.

[2] M. Bie, Y. Liu, G. Li, J. Hong, and J. Li, "Real-time vehicle detection algorithm based on a lightweight you-only-look-once (YOLOv5n-L) approach," *Expert Systems with Applications*, vol. 213, p. 119108, 2023.

[3] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "Detrs beat YOLOs on real-time object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 16 965–16 974.

[4] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *European Conference on Computer Vision*, 2022, pp. 1–21.

[6] J. Cao, X. Weng, R. Khirodkar, J. Pang, and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 9686–9696.

[7] Y. Zhang, T. Wang, and X. Zhang, "MOTRv2: Bootstrapping end-to-end multi-object tracking by pretrained object detectors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 22 056–22 065.

[8] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng, "StrongSORT: Make DeepSORT great again," *IEEE Transactions on Multimedia*, vol. 25, pp. 8725–8737, 2023.

[9] I. Shafi, I. Hussain, J. Ahmad, P. W. Kim, G. S. Choi, I. Ashraf, and S. Din, "License plate identification and recognition in a non-standard environment using neural pattern matching," *Complex & Intelligent Systems*, vol. 8, pp. 3627–3639, 2022.

[10] Q. Ke *et al.*, "Automatic traffic monitoring using spatial transformer network and monocular depth estimation," in *2024 IEEE International Conference on Mobility, Operations, Services and Technologies*, 2024, pp. 72–79.

[11] C. Wei *et al.*, "Efficient license plate recognition in unconstrained scenarios," *Journal of Visual Communication and Image Representation*, 2024, online publication.

[12] Y. Zhang *et al.*, "Emission analysis based on mixed traffic flow and license plate recognition model," *Transportation Research Part D: Transport and Environment*, vol. 134, p. 104331, 2024.

[13] C. S. Ong, T. Connie, and M. K. O. Goh, "Traffic violation detection using computer vision techniques," *International Journal on Informatics Visualization*, vol. 8, no. 3-2, pp. 1515–1527, 2024.

[14] M. M. Fahrezi and E. A. Laksana, "Traffic flow and congestion detection with YOLOv8 and ByteTrack-based multi object tracking," *Jurnal Teknik Informatika*, vol. 5, no. 4, 2024.

[15] S. Ravi *et al.*, "Comprehensive study on the development of an automatic helmet violator detection system using advanced machine learning techniques," *Computers & Electrical Engineering*, 2024, article 109227.

[16] H. Yang, J. Cai, C. Liu, R. Ke, and Y. Wang, "Cooperative multi-camera vehicle tracking and traffic surveillance with edge artificial intelligence and representation learning," *Transportation Research Part C: Emerging Technologies*, vol. 148, p. 103982, 2023.

[17] A. Basalamah *et al.*, "FrameHopper: Selective processing of video frames in detection-driven real-time video analytics," in *2022 18th International Conference on Distributed Computing in Sensor Systems*, 2022, pp. 125–132.

[18] Q. Zhou, S. Guo, J. Pan, J. Liang, Z. Xu, and J. Zhou, "PASS: Patch automatic skip scheme for efficient real-time video perception on edge devices," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 3, 2023, pp. 3787–3795.

[19] Y. Li, Y. Li, Y. Chen, J. Tong, X. Tian, and K. Chi, "Online resolution adaptation and resource allocation for edge-assisted video analytics," *Computer Networks*, vol. 249, p. 110342, 2024.

[20] X. Zhang *et al.*, "SplitStream: Distributed and workload-adaptive video analytics at the edge," *Journal of Network and Computer Applications*, vol. 225, p. 103866, 2024.